A.jashwanti
Roll No: 26000007
CSE-1

41. Nested List Classroom Occupancy Analysis

1. Problem Statement

To manage classroom seating using nested lists, allocate students, preserve reserved seats, identify duplicate assignments and calculate vacant seats.

2. Objectives

• Store classroom seating details.

• Allocate students to vacant seats.

• Preserve reserved seats.

• Detect duplicate student assignments.

• Calculate vacant capacity.

3. Assumptions

• There are 2 classrooms, each with 5 seats.

• 'R' represents a reserved seat; None represents a vacant seat.

4. Algorithm

1. Create nested lists for classrooms.

2. Allocate each student to the first available vacant seat, skipping reserved seats.

3. Check assigned names for duplicates.

4. Count vacant seats and display the report.

5. Flowchart

Start → Create classroom lists → Allocate students → Skip reserved/occupied seats → Check duplicates → Count vacant seats → Display report → End

6. Pseudocode

```
START
Create classroom nested list
FOR each student
    Find a vacant seat
    IF seat is available
        Allocate student
    ELSE
        Display seat unavailable
Check duplicate names
Count vacant seats
Display classroom details
STOP
```

7. Python Source Code

```
classrooms = [
    ["A", None, "R", None, "B"],
    [None, "C", None, "D", "R"]
]

students = ["Teja", "Ravi", "Priya"]

for student in students:
    allocated = False
    for room in classrooms:
        for i in range(len(room)):
            if room[i] is None:
                room[i] = student
                allocated = True
                break
        if allocated:
            break

print("Classroom Occupancy:")
for i, room in enumerate(classrooms):
```

```
    print("Classroom", i + 1, ":", room)

all_students = []
for room in classrooms:
    for seat in room:
        if seat is not None and seat != "R":
            all_students.append(seat)

duplicates = [s for s in set(all_students)
              if all_students.count(s) > 1]
vacant = sum(room.count(None) for room in classrooms)

print("Duplicate Students:", duplicates)
print("Vacant Seats:", vacant)
```

8. Test Cases

Test input	Expected result
3 students	Allocate 3 seats
Reserved seat	Remains reserved
Repeated name	Duplicate detected

9. Sample Output

```
=============== RESTART: C:/Users/arjaj/Documents/hello world.py ===============

Classroom Occupancy:
Classroom 1 : ['A', 'Teja', 'R', 'Ravi', 'B']
Classroom 2 : ['Priya', 'C', None, 'D', 'R']
Duplicate Students: []
Vacant Seats: 1
```

10. Output Screenshot Reference

The block above is an IDLE-style expected-output representation, not a captured screenshot. Run the program in Python IDLE and use Win + Shift + S to capture the actual Shell output for submission.

Explanation

The nested list stores each classroom's seats. The program fills only None positions, leaves reserved seats unchanged, checks repeated names and counts the remaining vacant seats.

42. Student Marks Matrix Analysis

1. Problem Statement

To analyse student marks using a matrix and calculate totals, averages, top performers and subjects with low average marks.

2. Objectives

• Store marks in a matrix.

• Calculate student totals and averages.

• Calculate subject averages.

• Identify the top performer.

• Detect subjects with low averages.

3. Assumptions

• There are 3 students and 3 subjects.

• Marks are out of 100; a subject average below 40 is considered low.

4. Algorithm

1. Create a marks matrix and student/subject names.

2. Calculate each student's total and average.

3. Find the student with the highest total.

4. Calculate each subject's average and identify averages below 40.

5. Display the report.

5. Flowchart

Start → Enter marks matrix → Calculate student totals/averages → Find top performer → Calculate subject averages → Check low averages → Display report → End

6. Pseudocode

```
START
Create marks matrix
FOR each student
    Calculate total and average
FOR each subject
    Calculate subject average
Find highest-scoring student
Find subjects below 40 average
Display results
STOP
```

7. Python Source Code

```
marks = [
    [85, 90, 78],
    [70, 65, 80],
    [95, 92, 96]
]

names = ["Teja", "Ravi", "Priya"]
subjects = ["Python", "Maths", "English"]

totals = []
for row in marks:
    totals.append(sum(row))

averages = [total / 3 for total in totals]

for i in range(3):
    print(names[i], "Total:", totals[i],
          "Average:", round(averages[i], 2))

top = totals.index(max(totals))
```

```
print("Top Performer:", names[top])

for j in range(3):
    avg = sum(marks[i][j] for i in range(3)) / 3
    print(subjects[j], "Average:", round(avg, 2))
    if avg < 40:
        print("Low Performing Subject:", subjects[j])
```

8. Test Cases

Test input	Expected result
Teja: 85, 90, 78	Total = 253
Priya: 95, 92, 96	Top performer = Priya
Subject average below 40	Low subject identified

9. Sample Output

```
=============== RESTART: C:/Users/arjaj/Documents/hello world.py ===============

Teja Total: 253 Average: 84.33
Ravi Total: 215 Average: 71.67
Priya Total: 283 Average: 94.33
Top Performer: Priya
Python Average: 83.33
Maths Average: 82.33
English Average: 84.67
```

10. Output Screenshot Reference

The block above is an IDLE-style expected-output representation, not a captured screenshot. Run the program in Python IDLE and use Win + Shift + S to capture the actual Shell output for submission.

Explanation

Rows represent students and columns represent subjects. Row calculations give student totals and averages; column calculations give subject averages.

43. Multi-Branch Sales Matrix Comparison

1. Problem Statement

To store product sales across branches using nested lists, calculate product totals, compare branch sales and identify the best-performing product in each branch.

2. Objectives

• Store sales data in a matrix.

• Calculate product-wise totals.

• Compare branch performance.

• Identify the best-selling product in each branch.

3. Assumptions

• There are 3 products and 3 branches.

• Each matrix value represents sales of one product in one branch.

4. Algorithm

1. Create the sales matrix and product/branch names.

2. Calculate each product's total sales.

3. Calculate each branch's total sales.

4. Find the product with the highest sales in each branch.

5. Display the comparison.

5. Flowchart

Start → Create sales matrix → Calculate product totals → Calculate branch totals → Find best product per branch → Display comparison → End

6. Pseudocode

```
START
Create sales matrix
Calculate product totals
Calculate branch totals
FOR each branch
    Find highest-selling product
Display sales report
STOP
```

7. Python Source Code

```
sales = [
    [100, 150, 200],
    [250, 200, 180],
    [150, 300, 250]
]

products = ["Laptop", "Mobile", "Tablet"]
branches = ["Branch A", "Branch B", "Branch C"]

for i in range(3):
    print(products[i], "Total:", sum(sales[i]))

for j in range(3):
    total = sum(sales[i][j] for i in range(3))
    best = max(range(3), key=lambda i: sales[i][j])
    print(branches[j], "Sales:", total)
    print("Best Product:", products[best])
```

8. Test Cases

Test input	Expected result
Laptop sales: 100, 150, 200	Total = 450

Test input	Expected result
Branch B sales	Total = 650
Branch A best product	Mobile

9. Sample Output

```
=============== RESTART: C:/Users/arjaj/Documents/hello world.py ===============

Laptop Total: 450
Mobile Total: 630
Tablet Total: 700
Branch A Sales: 500
Best Product: Mobile
Branch B Sales: 650
Best Product: Tablet
Branch C Sales: 630
Best Product: Tablet
```

10. Output Screenshot Reference

The block above is an IDLE-style expected-output representation, not a captured screenshot. Run the program in Python IDLE and use Win + Shift + S to capture the actual Shell output for submission.

Explanation

Rows represent products and columns represent branches. The program adds each row for product totals and each column for branch totals.

44. Hospital Ward Occupancy Monitoring

1. Problem Statement

To monitor hospital bed occupancy using nested lists, calculate utilization percentages, detect capacity violations and identify peak occupancy.

2. Objectives

• Store daily ward occupancy.

• Calculate utilization percentages.

• Identify capacity violations.

• Find peak occupancy.

3. Assumptions

• There are 3 wards with a capacity of 10 beds each.

• Each row contains occupancy for 3 days.

4. Algorithm

1. Create an occupancy matrix and ward capacities.

2. Calculate total occupied beds for each ward.

3. Calculate utilization as total occupancy divided by total available beds, multiplied by 100.

4. Check whether any daily occupancy exceeds capacity.

5. Find the peak occupancy and display the report.

5. Flowchart

Start → Enter occupancy matrix → Read capacities → Calculate utilization → Check capacity violations → Find peak occupancy → Display report → End

6. Pseudocode

```
START
Create occupancy matrix
Set ward capacities
FOR each ward
    Calculate total occupancy
    Calculate utilization percentage
    Check daily capacity violations
Find peak occupancy
Display report
STOP
```

7. Python Source Code

```
occupancy = [
    [7, 8, 9],
    [5, 12, 8],
    [6, 7, 9]
]

wards = ["General", "ICU", "Emergency"]
capacity = [10, 10, 10]

for i in range(3):
    total = sum(occupancy[i])
    percentage = (total / (capacity[i] * 3)) * 100

    print(wards[i], "Total Beds:", total)
    print("Utilization:", round(percentage, 2), "%")

    if max(occupancy[i]) > capacity[i]:
        print("Capacity Violation Detected")

peak = max(max(row) for row in occupancy)
print("Peak Occupancy:", peak)
```

8. Test Cases

Test input	Expected result
General: 7, 8, 9	Utilization = 80%
ICU: 5, 12, 8	Capacity violation
All wards	Peak occupancy = 12

9. Sample Output

```
=============== RESTART: C:/Users/arjaj/Documents/hello world.py ===============

General Total Beds: 24
Utilization: 80.0 %
ICU Total Beds: 25
Utilization: 83.33 %
Capacity Violation Detected
Emergency Total Beds: 22
Utilization: 73.33 %
Peak Occupancy: 12
```

10. Output Screenshot Reference

The block above is an IDLE-style expected-output representation, not a captured screenshot. Run the program in Python IDLE and use Win + Shift + S to capture the actual Shell output for submission.

Explanation

Each row stores a ward's daily occupancy. Utilization is calculated against the ward's total bed capacity over all recorded days. A daily value above capacity is flagged.

45. Weekly Electricity Consumption Matrix

1. Problem Statement

To analyse appliance-wise electricity consumption for seven days, calculate daily and appliance totals and identify consumption exceeding a specified limit.

2. Objectives

• Store weekly electricity readings.

• Calculate appliance-wise totals.

• Calculate daily totals.

• Identify high consumption.

3. Assumptions

• There are 3 appliances and 7 days.

• The daily consumption limit is 15 units.

4. Algorithm

1. Create the weekly consumption matrix.

2. Calculate the total for each appliance.

3. Calculate the total for each day.

4. Compare each daily total with 15 units.

5. Display warnings and totals.

5. Flowchart

Start → Enter weekly readings → Calculate appliance totals → Calculate daily totals → Compare with limit → Display warnings/report → End

6. Pseudocode

```
START
Create consumption matrix
Calculate appliance totals
FOR each day
    Calculate daily consumption
    IF consumption > limit
        Display warning
Display weekly totals
STOP
```

7. Python Source Code

```
usage = [
    [2, 3, 2, 4, 3, 2, 3],
    [4, 4, 5, 4, 5, 4, 4],
    [3, 3, 4, 3, 4, 3, 3]
]

appliances = ["Fan", "AC", "Light"]

for i in range(3):
    print(appliances[i], "Total:", sum(usage[i]))

for j in range(7):
    daily = sum(usage[i][j] for i in range(3))
    print("Day", j + 1, ":", daily, "units")

    if daily > 15:
        print("High Consumption Alert")
```

8. Test Cases

Test input	Expected result
Fan readings	Total = 19 units
Day 1 readings: 2, 4, 3	Total = 9 units
Daily total above 15	High consumption alert

9. Sample Output

```
=============== RESTART: C:/Users/arjaj/Documents/hello world.py ===============

Fan Total: 19
AC Total: 30
Light Total: 23
Day 1 : 9 units
Day 2 : 10 units
Day 3 : 11 units
Day 4 : 11 units
Day 5 : 12 units
Day 6 : 9 units
Day 7 : 10 units
```

10. Output Screenshot Reference

The block above is an IDLE-style expected-output representation, not a captured screenshot. Run the program in Python IDLE and use Win + Shift + S to capture the actual Shell output for submission.

Explanation

Rows represent appliances and columns represent days. The program calculates row totals and column totals. The supplied sample data does not exceed the 15-unit daily limit, so no alert appears.

46. Inventory Matrix Reconciliation

1. Problem Statement

To compare product quantities across warehouses using nested lists, identify shortages and surpluses and calculate the difference from target stock.

2. Objectives

• Store warehouse inventory.

• Compare actual and target stock.

• Identify shortages and surpluses.

• Calculate stock differences.

3. Assumptions

• There are 2 warehouses and 3 products.

• The target stock is 20 units for each product in each warehouse.

4. Algorithm

1. Create the inventory matrix and product names.

2. Set the target stock quantity.

3. Compare each actual quantity with the target.

4. Report shortage, surplus or balanced stock.

5. Display the inventory report.

5. Flowchart

Start → Enter inventory matrix → Set target stock → Compare actual and target → Calculate shortage/surplus → Display report → End

6. Pseudocode

```
START
Create inventory matrix
Set target stock
FOR each warehouse
    FOR each product
        Compare actual and target
        Calculate shortage or surplus
Display inventory report
STOP
```

7. Python Source Code

```
inventory = [
    [25, 15, 30],
    [10, 25, 18]
]

products = ["Laptop", "Mouse", "Keyboard"]
target = 20

for i in range(2):
    print("Warehouse", i + 1)

    for j in range(3):
        difference = inventory[i][j] - target

        if difference < 0:
            print(products[j], "Shortage:", -difference)
        elif difference > 0:
            print(products[j], "Surplus:", difference)
        else:
            print(products[j], "Stock Balanced")
```

8. Test Cases

Test input	Expected result
Actual stock 25, target 20	Surplus = 5
Actual stock 15, target 20	Shortage = 5
Actual stock 20, target 20	Balanced

9. Sample Output

```
=============== RESTART: C:/Users/arjaj/Documents/hello world.py ===============

Warehouse 1
Laptop Surplus: 5
Mouse Shortage: 5
Keyboard Surplus: 10
Warehouse 2
Laptop Shortage: 10
Mouse Surplus: 5
Keyboard Shortage: 2
```

10. Output Screenshot Reference

The block above is an IDLE-style expected-output representation, not a captured screenshot. Run the program in Python IDLE and use Win + Shift + S to capture the actual Shell output for submission.

Explanation

The program compares every product quantity with the target of 20. A positive difference is a surplus and a negative difference is a shortage. This basic version reports differences; it does not automatically transfer stock.

47. Sports Tournament Ranking System

1. Problem Statement

To manage sports tournament results, calculate team points, apply score-difference tie-break rules and prepare team rankings.

2. Objectives

• Store match results.

• Calculate team points.

• Calculate score differences.

• Prepare final rankings.

3. Assumptions

• Three teams participate.

• Win = 3 points, draw = 1 point and loss = 0 points.

• Score difference is the tie-breaker after points.

4. Algorithm

1. Create team records for points and score difference.

2. Enter match results.

3. Award points based on each result.

4. Update score differences.

5. Sort teams by points and score difference in descending order.

6. Display rankings.

5. Flowchart

Start → Enter match results → Award points → Calculate score difference → Sort teams → Display rankings → End

6. Pseudocode

```
START
Create team records
FOR each match
    Compare scores
    Award points
    Update score difference
Sort by points and score difference
Display rankings
STOP
```

7. Python Source Code

```
teams = {
    "India": [0, 0],
    "Australia": [0, 0],
    "England": [0, 0]
}

matches = [
    ("India", "Australia", 3, 1),
    ("England", "India", 2, 2),
    ("Australia", "England", 4, 2)
]

for a, b, sa, sb in matches:
    teams[a][1] += sa - sb
    teams[b][1] += sb - sa

    if sa > sb:
        teams[a][0] += 3
    elif sb > sa:
        teams[b][0] += 3
```

```
    else:
        teams[a][0] += 1
        teams[b][0] += 1

ranking = sorted(
    teams.items(),
    key=lambda x: (x[1][0], x[1][1]),
    reverse=True
)

print("Tournament Rankings:")
for i, (name, values) in enumerate(ranking, 1):
    print(i, name, "Points:", values[0],
          "Difference:", values[1])
```

8. Test Cases

Test input	Expected result
India 3–1 Australia	India wins; 3 points
England 2–2 India	Both receive 1 point
Australia 4–2 England	Australia wins; 3 points

9. Sample Output

```
=============== RESTART: C:/Users/arjaj/Documents/hello world.py ===============

Tournament Rankings:
1 India Points: 4 Difference: 2
2 Australia Points: 3 Difference: 0
3 England Points: 1 Difference: -2
```

10. Output Screenshot Reference

The block above is an IDLE-style expected-output representation, not a captured screenshot. Run the program in Python IDLE and use Win + Shift + S to capture the actual Shell output for submission.

Explanation

The program awards points after each match and updates each team's score difference. Teams are ranked by points first and score difference second.

48. Tuple-Based Employee Shift Conflict Detection

1. Problem Statement

To store employee shift schedules using tuples, compare time intervals and identify overlapping shifts assigned to the same employee.

2. Objectives

• Store employee shift details.

• Compare shift timings.

• Detect overlapping shifts.

• Display conflicting schedules.

3. Assumptions

• Each tuple contains employee name, start time and end time.

• Times use the 24-hour clock as whole numbers.

• Shifts overlap when start1 < end2 and start2 < end1 for the same employee.

4. Algorithm

1. Create shift tuples.

2. Compare every pair of shifts.

3. Check whether the employee names match.

4. Check whether the time intervals overlap.

5. Display any conflicts.

5. Flowchart

Start → Create shift tuples → Compare pairs → Same employee? → Check time overlap → Report conflict → End

6. Pseudocode

```
START
Create employee shift tuples
FOR each pair of shifts
    IF employee names are the same
        IF start1 < end2 AND start2 < end1
            Display conflict
STOP
```

7. Python Source Code

```
shifts = [
    ("Ravi", 9, 13),
    ("Teja", 10, 14),
    ("Ravi", 12, 16),
    ("Priya", 14, 18)
]

found = False

for i in range(len(shifts)):
    for j in range(i + 1, len(shifts)):
        name1, start1, end1 = shifts[i]
        name2, start2, end2 = shifts[j]

        if name1 == name2:
            if start1 < end2 and start2 < end1:
                print("Conflict:", name1)
                print("Shift 1:", start1, "-", end1)
                print("Shift 2:", start2, "-", end2)
                found = True

if not found:
    print("No Conflicts Found")
```

8. Test Cases

Test input	Expected result
Ravi: 9–13 and 12–16	Conflict detected
Teja: one shift only	No conflict
Different employees with overlapping times	No same-employee conflict

9. Sample Output

```
=============== RESTART: C:/Users/arjaj/Documents/hello world.py ===============

Conflict: Ravi
Shift 1: 9 - 13
Shift 2: 12 - 16
```

10. Output Screenshot Reference

The block above is an IDLE-style expected-output representation, not a captured screenshot. Run the program in Python IDLE and use Win + Shift + S to capture the actual Shell output for submission.

Explanation

Each shift is stored as a tuple. The program compares pairs only when the employee names match, then checks whether their time intervals overlap.

49. Tuple-Based Product Catalogue Audit

1. Problem Statement

To maintain product catalogue records using tuples, identify invalid records, calculate inventory values and find products requiring replenishment.

2. Objectives

• Store product details using tuples.

• Identify invalid prices and quantities.

• Calculate inventory value.

• Identify low-stock products.

3. Assumptions

• Each tuple contains product ID, price and stock quantity.

• Price must be greater than zero; stock must be zero or greater.

• Replenishment is required when stock is below 10 units.

4. Algorithm

1. Create product tuples.

2. Read each product record.

3. Validate price and stock.

4. Calculate inventory value for valid records.

5. Identify products with stock below 10.

6. Display the report.

5. Flowchart

Start → Create product tuples → Validate record → Calculate inventory value → Check low stock → Display report → End

6. Pseudocode

```
START
Create product tuples
FOR each product
    Check price and stock
    IF invalid
        Display invalid record
    ELSE
        Calculate inventory value
        Check replenishment level
Display results
STOP
```

7. Python Source Code

```
products = [
    ("P101", 500, 20),
    ("P102", 300, 5),
    ("P103", -100, 10),
    ("P104", 700, 0)
]

total_value = 0

for pid, price, stock in products:
    if price <= 0 or stock < 0:
        print(pid, "Invalid Record")
        continue

    value = price * stock
    total_value += value

    print(pid, "Inventory Value:", value)
```

```
    if stock < 10:
        print("Replenishment Required")

print("Total Inventory Value:", total_value)
```

8. Test Cases

Test input	Expected result
Price 500, stock 20	Inventory value = 10000
Price -100	Invalid record
Stock 5	Replenishment required

9. Sample Output

```
=============== RESTART: C:/Users/arjaj/Documents/hello world.py ===============

P101 Inventory Value: 10000
P102 Inventory Value: 1500
Replenishment Required
P103 Invalid Record
P104 Inventory Value: 0
Replenishment Required
Total Inventory Value: 11500
```

10. Output Screenshot Reference

The block above is an IDLE-style expected-output representation, not a captured screenshot. Run the program in Python IDLE and use Win + Shift + S to capture the actual Shell output for submission.

Explanation

Each product tuple stores its ID, price and quantity. Valid records contribute price multiplied by stock to the inventory value. Records with invalid price or negative stock are skipped.

50. Tuple-Based Route Distance Analysis

1. Problem Statement

To store geographical coordinates using tuples, calculate distances between successive locations using the Euclidean distance formula and identify the longest route segment.

2. Objectives

• Store geographical coordinates.

• Calculate distances between successive locations.

• Identify the longest route segment.

• Calculate total route distance.

3. Assumptions

• Coordinates are represented as (x, y) tuples.

• The Euclidean distance formula is used.

• Coordinates are measured in kilometres on a simplified plane.

4. Algorithm

1. Create coordinate tuples.

2. Select each pair of successive locations.

3. Calculate distance using the Euclidean formula.

4. Add each distance to the total.

5. Find the longest segment and display results.

5. Flowchart

Start → Enter coordinate tuples → Select successive points → Calculate distance → Add to total → Find longest segment → Display results → End

6. Pseudocode

```
START
Create coordinate tuples
Set total distance = 0
FOR each successive point pair
    Calculate Euclidean distance
    Add distance to total
    Compare with longest distance
Display total distance
Display longest segment
STOP
```

7. Python Source Code

```
import math

routes = [
    (0, 0),
    (3, 4),
    (6, 8),
    (12, 8)
]

distances = []

for i in range(len(routes) - 1):
    x1, y1 = routes[i]
    x2, y2 = routes[i + 1]

    distance = math.sqrt(
        (x2 - x1)**2 + (y2 - y1)**2
    )

    distances.append(distance)
```

```
    print("Segment", i + 1, "Distance:",
          round(distance, 2), "km")

print("Total Distance:", round(sum(distances), 2), "km")

longest = max(distances)
index = distances.index(longest) + 1

print("Longest Segment:", index)
print("Longest Distance:", round(longest, 2), "km")
```

8. Test Cases

Test input	Expected result
(0,0) to (3,4)	5 km
(3,4) to (6,8)	5 km
(6,8) to (12,8)	6 km

9. Sample Output

```
=============== RESTART: C:/Users/arjaj/Documents/hello world.py ===============

Segment 1 Distance: 5.0 km
Segment 2 Distance: 5.0 km
Segment 3 Distance: 6.0 km
Total Distance: 16.0 km
Longest Segment: 3
Longest Distance: 6.0 km
```

10. Output Screenshot Reference

The block above is an IDLE-style expected-output representation, not a captured screenshot. Run the program in Python IDLE and use Win + Shift + S to capture the actual Shell output for submission.

Explanation

The Euclidean formula calculates the straight-line distance between two coordinate points. The program adds successive segment distances and identifies the largest one.